

Multi – level Linked List

–Destroy List(Main List and Child)

–Time Complexity Calculation using Substitution Method

Program:

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

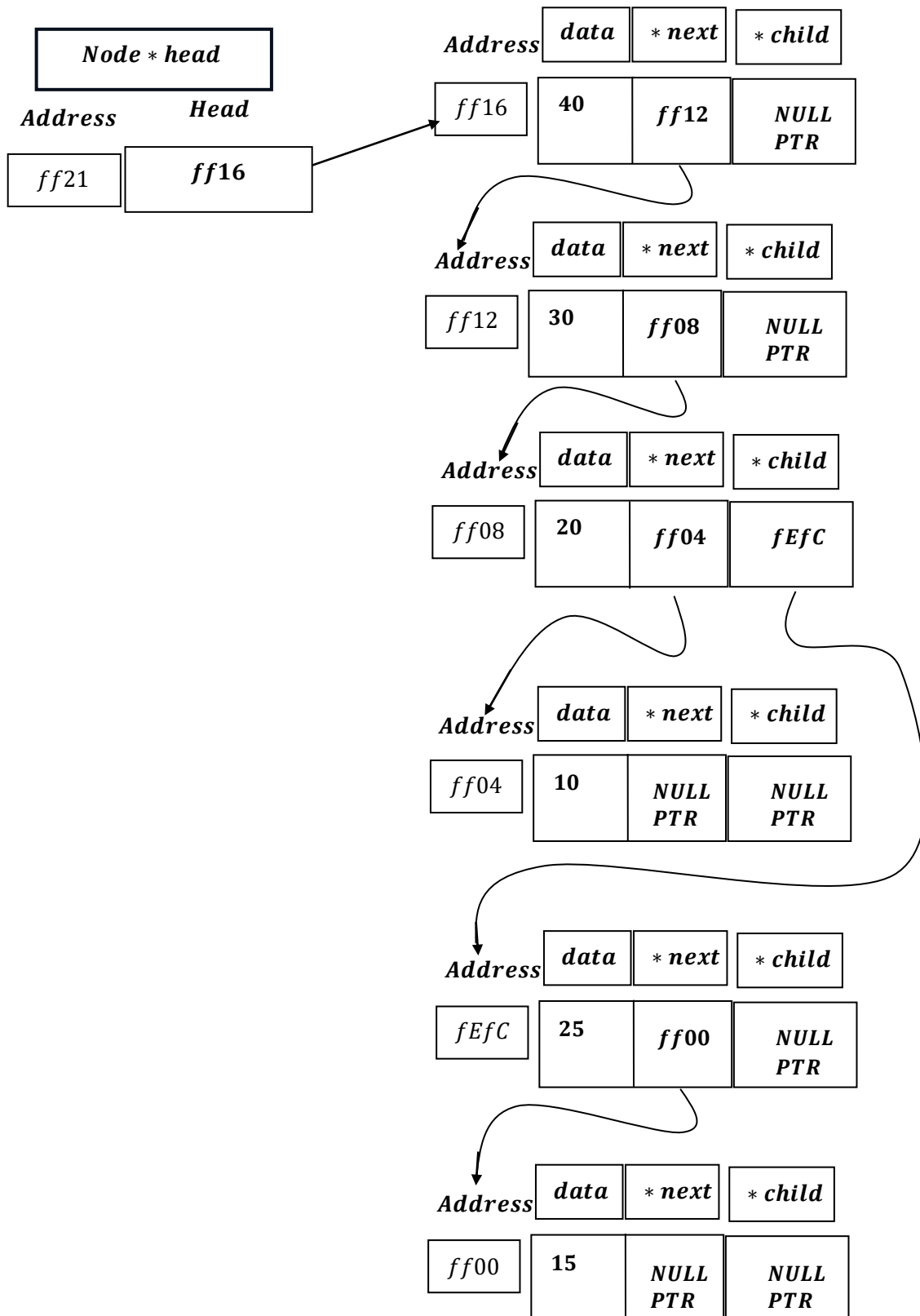
    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}

void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}

int main()
{
    int choice;
    while (true)
    {
        cout << "\n--- Multilevel List Menu ---" << endl;
        cout << "8. Destroy List" << endl;
        cin >> choice;
```

```
        switch (choice)
        {
        case 8:
            destroyList();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```



Here , we get total amount of nodes in the list : 06 .

No. of child nodes : -2

destroyNodeRecursively(node → child: 2 nodes)

say, we take 1 node to gets deleted .

Then: Remaining nodes : 6 - 1 (generally)

But in Next List without child node ,we have: 6 - 1 - 2 = 3 nodes.

destroyNodeRecursively(node → next: 3 nodes), after

1 node already considered to get deleted executed by free(node).

So, 2 + 3 + 1 = 6 nodes.

Total number of Nodes: N and Time Complexity for nodes `N` is : T(N).

Lets say, total child nodes is `k` ,to be deleted by T(k) time.

Then if 1 node considered to be deleted by free(node)take constant ,

time complexity : `c` .

Then on next list we will get : T(N - 1 - k)

Therefore ,General Equation for time complexity calculation:

$$T(N) = T(k) + T(N - 1 - k) + c$$

The base case occurs when the function encounters a null pointer, at which point it immediately returns.

$$T(0) = O(1)$$

$$T(N) = \begin{cases} 1 & , N = 0 \\ T(k) + T(N - 1 - k) + c & , N > 0 \end{cases}$$

Similarly, we have to know the behaviour of `k` .

We know from above analogy that:

$$k + (N - 1 - k) + 1 = N$$

Then , `k` must have quality :

$$0 \leq k \leq N - 1$$

i. e. $k + 1 = N$ and $N - 1 - k = 0$

in both the cases , we get , $k = N - 1$

$\therefore k \neq N$.

It gives us two prospective:

[Worst Structure]

1) $k = 0$, will generate Unbalanced Tree.

which means:

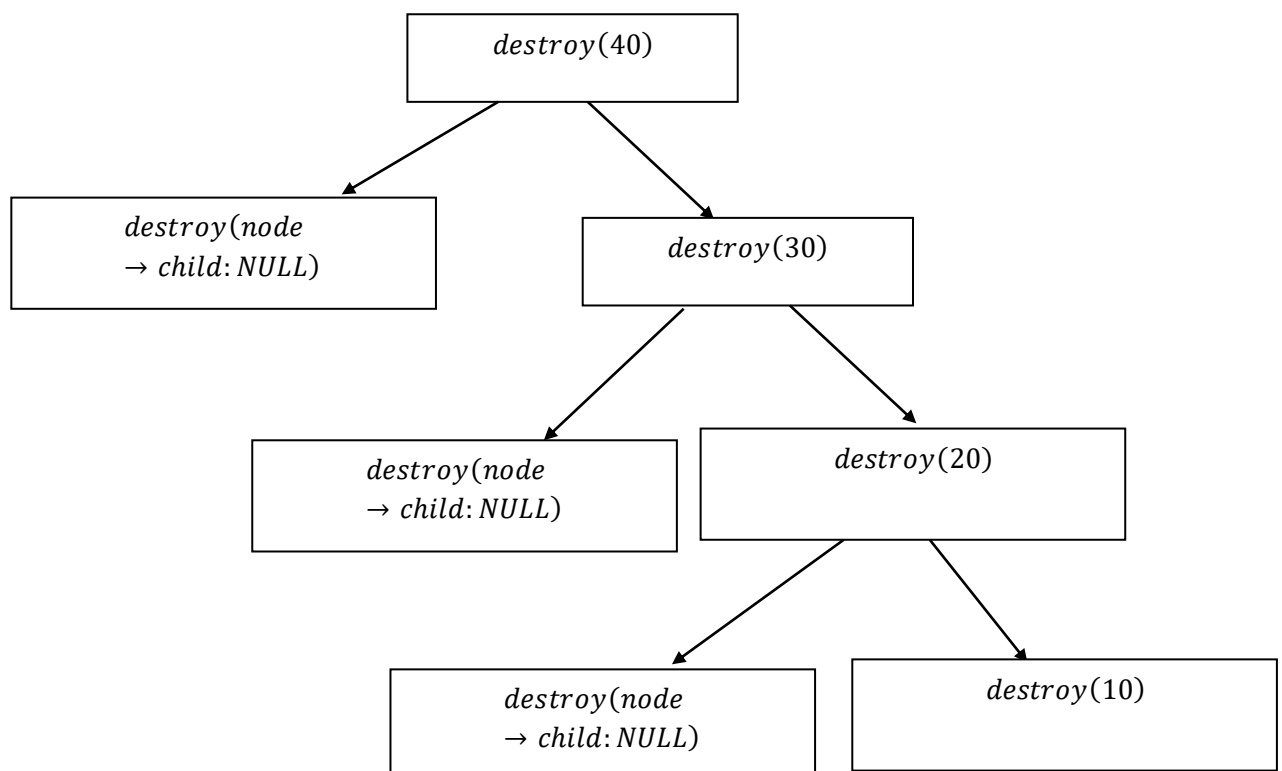
- *child branch has 0 nodes*
- *next branch has $N - 1$ nodes*

So recursion expands only in ONE direction.

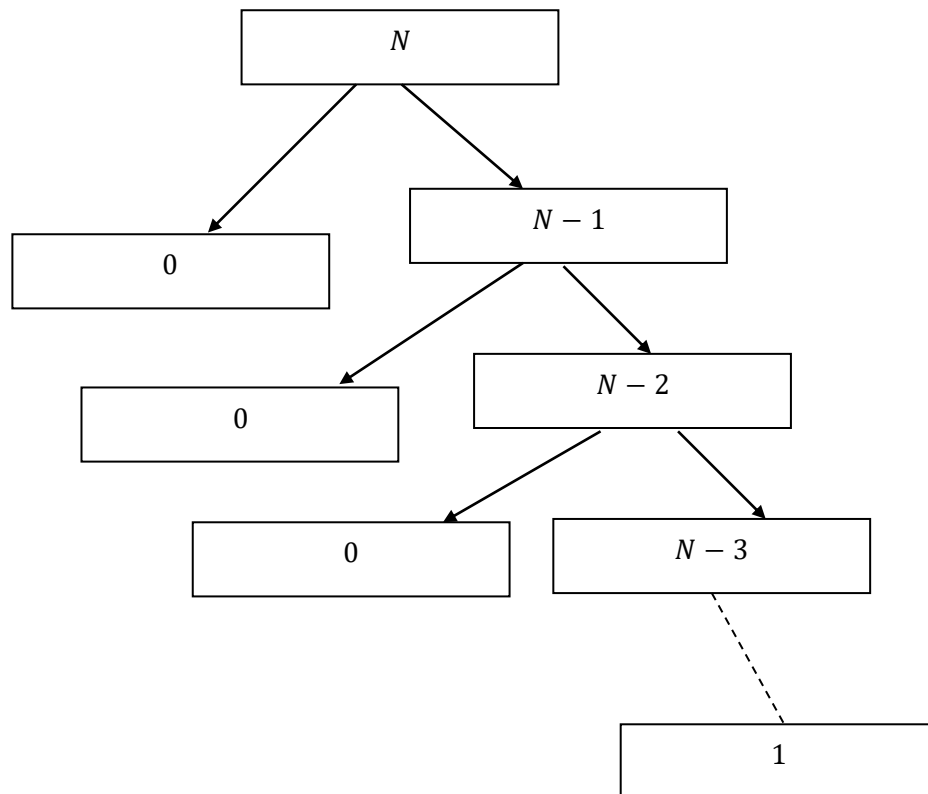
$$T(N) = T(0) + T(N - 1 - 0) + C$$

We can combine $T(0)$ and C into a single constant `C` .

$$\Rightarrow T(N) = T(N - 1) + C$$



*of a list contain only next : $40 \rightarrow 30 \rightarrow 20 \rightarrow 10$
And 0 child nodes.*



As , $NULLPTR$ represents 0.

Another Worst Structure

2) $k = N - 1$, will generate *Unbalanced Tree*.

which means:

- *child branch has $N - 1$ nodes*
- *next branch has 0 nodes*

So recursion expands only in ONE direction.

$$T(N) = T(k) + T(N - 1 - k) + c, N > 0$$

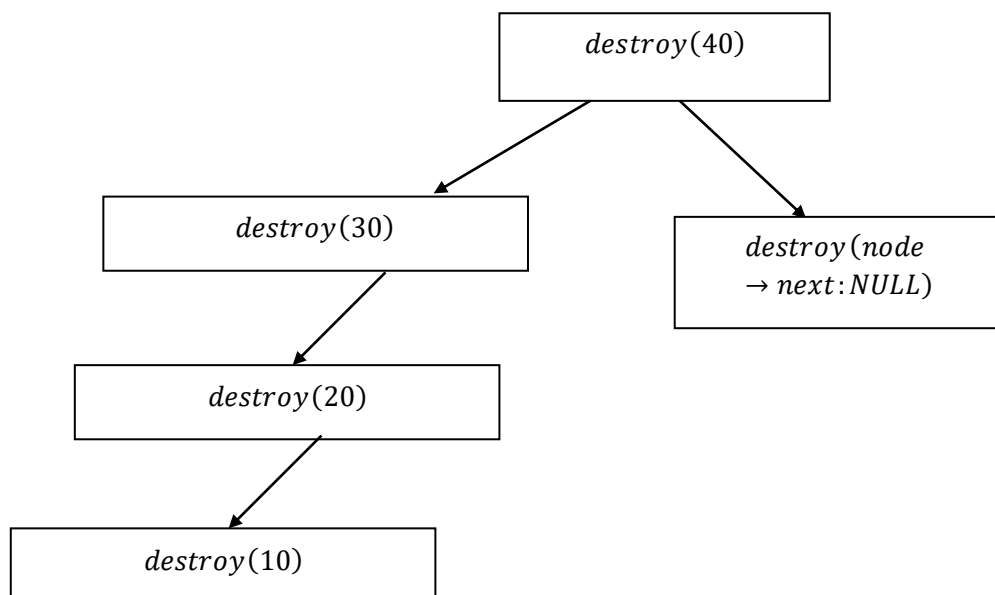
$$T(N) = T(N - 1) + T(N - 1 - (N - 1)) + c$$

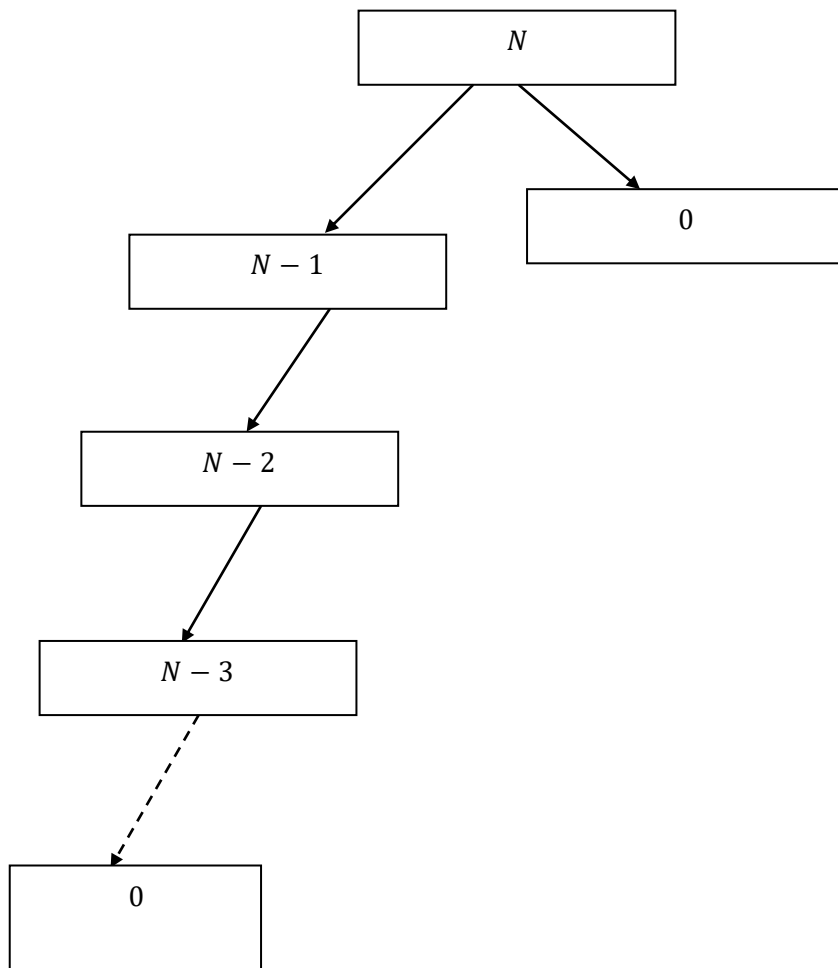
$$T(N) = T(N - 1) + T(N - 1 - N + 1) + c$$

$$T(N) = T(N - 1) + T(0) + c$$

We can combine $T(0)$ and C into a single constant `C` .

$$\Rightarrow T(N) = T(N - 1) + C$$





[Balanced Structure]

2) $k = N - 1$, divided into 2 equal halves will generate Balanced Tree .

Say 1 node is already taken for $free(node)$ execution.

- **The child path happens to have $\frac{N-1}{2}$ nodes attached to it $\left(k = \frac{N-1}{2}\right)$.**
- **The next path happens to have the other $\left(\frac{N-1}{2}\right)$ nodes.**

Through Mathematically, we can derive:

$$k(\text{No. of Child Node}) = (N - 1 - k) \text{ [Other Half]}$$

$$k = N - 1 - k$$

$$2k = N - 1$$

$$k = \frac{N - 1}{2}$$

Now, divided into two halves : $2T\left(\frac{N-1}{2}\right) + C$ (for execution of $free(node)$).

$$\text{i. e.: } 2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

[Worst Structure] – By Substitution Method

$$T(N) = T(N - 1) + c$$

$$\therefore T(N - 1) = T(N - 2) + c$$

Substituting in $T(N) = T(N - 1) + c$, we get:

$$\Rightarrow T(N) = T(N - 2) + c + c \Rightarrow T(N - 2) + 2c$$

$$\therefore T(N - 2) = T(N - 3) + c$$

Substituting in $T(N) = T(N - 2) + 2c$, we get:

$$\Rightarrow T(N) = T(N - 3) + 2c + c \Rightarrow T(N - 3) + 3c$$

if it goes for `i` time ,we get:

$$\Rightarrow T(N) = T(N - i) + ic$$

As , we see $T(0) = 1$,i. e. Base Case when $T(0)$.

$$\therefore N - i = 0 \text{ [0 for } T(0)]$$

$$\therefore i = N$$

$$\Rightarrow T(N) = T(N - N) + N \cdot c$$

$$\Rightarrow T(N) = T(0) + N \cdot c$$

As, $T(0)$ is constant, we can write it as: $T(0) = c_0$

$$\Rightarrow T(N) = c_0 + N \cdot c$$

By applying Big - O, removing all constants we get:

$$\Rightarrow T(N) = O(N)$$

[Balanced Structure] – By Substitution Method.

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

$$2T\left(\frac{N-1}{2}\right) + C, \text{ can also be written as : } 2T\left(\frac{N-(2-1)}{2}\right) + C$$

$$\begin{aligned} \text{Now, } T\left(\frac{N-1}{2}\right) &= 2T\left(\frac{\frac{N-1}{2}-1}{\frac{2}{1}}\right) + C = 2T\left(\frac{\frac{N-1-2}{2}}{\frac{2}{1}}\right) + C \\ &= 2T\left(\frac{N-3}{2} \times \frac{1}{2}\right) + C \\ &= 2T\left(\frac{N-3}{4}\right) + C \end{aligned}$$

$$\text{Substituting it in : } 2T\left(\frac{N-1}{2}\right) + C$$

$$\Rightarrow 2\left[2T\left(\frac{N-3}{4}\right) + C\right] + C$$

$$\Rightarrow 2^2T\left(\frac{N-3}{2^2}\right) + 2C + C$$

$$\Rightarrow 2^2T\left(\frac{N-(2^2-1)}{2^2}\right) + 2C + C$$

$$\begin{aligned}
 \text{Now, } T\left(\frac{N-3}{4}\right) &= 2T\left(\frac{\frac{N-3}{4}-1}{\frac{2}{1}}\right) + C = 2T\left(\frac{\frac{N-4-3}{4}}{\frac{2}{1}}\right) + C \\
 &= 2T\left(\frac{N-7}{8}\right) + C
 \end{aligned}$$

$$\text{Substituting it in : } 2^2 T\left(\frac{N-3}{4}\right) + 2C + C$$

$$\Rightarrow 2^2 \left[2T\left(\frac{N-7}{8}\right) + C \right] + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N-7}{2^3}\right) + 2^2 C + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N - (2^3 - 1)}{2^3}\right) + 2^2 C + 2C + C$$

Thus ,if it runs upto `k` times we get:

$$\Rightarrow 2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + 2^{k-1}C + \dots + 2^2 C + 2C + C$$

$$\Rightarrow 2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + C (2^{k-1} + \dots + 2^2 + 2^1 + 2^0)$$

Now , here $2^{k-1} + \dots + 2^2 + 2^1 + 2^0$ is geometric progression.

Where number of terms counting from $2^0, 2^1, \dots, 2^{k-1} = k$ terms

We can also find it out by:

Number of integers from a to $b = b - a + 1$.

Where b is ending index.

a is starting index.

Here, $b = k - 1$ and $a = 0$.

$(k - 1) - 0 + 1 = k$ terms.

By formula : $S_n = a \left(\frac{r^n - 1}{r - 1} \right)$, where $a =$ starting value : $2^0 = 1$

$r = 2$

$n =$ number of terms $= k$

$$S_n = 2^0 \left(\frac{2^k - 1}{2 - 1} \right) = 1 \left(\frac{2^k - 1}{1} \right) = 2^k - 1.$$

$$\therefore 2^k T \left(\frac{N - (2^k - 1)}{2^k} \right) + C(2^k - 1)$$

As , we know, $T(0) = 1$, i. e. base case at 0 then:

$$\Rightarrow \frac{N - (2^k - 1)}{2^k} = 0$$

$$\Rightarrow N - (2^k - 1) = 0$$

$$\Rightarrow N - 2^k + 1 = 0$$

$$\Rightarrow N + 1 = 2^k$$

Applying , $2^k = N + 1$ in formula : $2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + C(2^k - 1)$,

We get:

$$\Rightarrow (N + 1) \times T\left(\frac{N - ((N + 1) - 1)}{N + 1}\right) + C((N + 1) - 1)$$

$$\Rightarrow (N + 1) \times T\left(\frac{N - N}{N + 1}\right) + C(N)$$

$$\Rightarrow (N + 1) \times T\left(\frac{0}{N + 1}\right) + C(N)$$

$$\Rightarrow (N + 1) \times T(0) + C(N) \left[\begin{array}{l} \text{since } \frac{0}{a} = 0, \text{ where } a > 0 \\ \text{As } N + 1, \therefore N \geq 0. \end{array} \right]$$

As, $T(0)$ is constant , let $T(0) = c_0$, knowing $T(0) = 1$, and we can implement `1` instead of $T(0)$ and end the process and we can also replace `C` with 1 (as it is a standard) but lets keep it c_0 and continue:

$$\Rightarrow (N + 1) \times c_0 + c \times N [C \text{ or } c \text{ represents same constants}]$$

By applying Big - O , removing all constants we get:

$$\Rightarrow O((N + 1)) + O(N)$$

$$\Rightarrow O(N)$$

Note:

The recurrence: $2T\left(\frac{N-1}{2}\right) + C$

arises from a balanced recursive structure of the multilevel linked list, but the algorithm itself is fundamentally recursive traversal rather than true divide – and – conquer.

Actual algorithm, it is NOT truly divide – and – conquer in the algorithmic sense.

The recurrence appears only because of a possible balanced recursion shape.

Hence, Real algorithm of Destroy List is NOT true divide – and – conquer.

$\therefore 2T\left(\frac{N-1}{2}\right) + C$, this one is hypothetically assumed based on balanced recursive structure.
